



NAME: MURILLO, SANDY P.
COURSE, YEAR & SECTION: BSIS 2A

SUBJECT: IS ELEC 2
PROFESSOR: REYMARK LLAGAS

TROUBLESHOOTING EXERCISE

SCENARIO	PROBLEM	CORRECT FIX
1	Used \$_POST , but the ID is literally from the URL.	Use \$_GET and see if the value shows up.
2	Missed the quotation marks for a string in the SQL code.	Put the first name in quotes.
3	Using \$_GET['age'] directly can open you up to SQL injection.	Use a prepared statement so the age gets properly bound.
4	Blank fields are being inserted due to the absence of validation.	Make sure the first and last name fields are not empty before insertion.
5	There's a typo in the POST key — it says 'emial' instead of 'email'.	Use 'email' for the POST key instead.
6	Using raw GET input for DELETE is risky.	Use a prepared statement or convert the ID to a number first.
7	Email has no quotes and errors aren't being handled.	Add quotes around the value and check if the query worked.
8	No loop means only the first row is fetched.	Implement a loop to iterate over and fetch all rows.
9	POST is used, yet the link actually sends GET.	Switch it to \$_GET .
10	The variable (\$aeg) is incorrectly named.	Change the variable to \$age.
11	Form method is GET, but the server expects POST.	Make the form and PHP use the same method.
12	The numeric ID is put inside quotes.	Cast the ID to int or just remove the quotes.
13	The UPDATE statement doesn't have a WHERE clause.	Add WHERE so only the chosen student gets updated.
14	Incorrect POST usage; need quotes around the key.	Correct the array access and treat the data like strings.
15	Lack of validation on the page number allows inappropriate or extremely high values.	Ensure page is an integer and limit it to valid values.

EXPLANATION:

SCENARIO 1: Code problem: expecting POST pero GET ang nandoon sa link. Use **\$_GET** para ma-capture ng tama yung parameter.

SCENARIO 2: Dapat naka-quote ang strings sa SQL. Pag wala, column name ang iisipin ng SQL, kaya nagkaka-error na 'unknown column.

SCENARIO 3: "Direct GET values sa SQL is super risky kasi pwede kang ma-SQL inject. Prepared statements ensure na safe at sanitized muna yung data bago mag run yung query.

SCENARIO 4: Blank inputs can mess up your database, kaya mag-validate muna bago mag-INSERT.

SCENARIO 5: Wrong POST key means PHP can't find the value, kaya undefined index yung error. Just fix the key para makuha yung email nang walang issue.

SCENARIO 6: If direct GET value ang gamit sa DELETE, kaya niyang mag-delete ng sobra-sobrang rows. Pero pag ginawa mo siyang integer, limited ka lang sa isang valid ID.

SCENARIO 7: Even if SQL fails, script still says 'Updated!' kasi hindi niya chine-check yung errors. Add proper error handling para accurate ang report.

SCENARIO 8: Without a loop, `mysqli_fetch_assoc` will give only one row. Use a while loop to fetch every record.

SCENARIO 9: Link triggers GET request pero script reads POST, so nagkaka-error. Use **\$_GET** para aligned sila.

SCENARIO 10: Wrong variable name causes PHP warning at baka mag-break yung SQL. Correct the variable para ma-access ng tama yung value.

SCENARIO 11: Form and PHP method mismatch: GET vs POST. Fix it para parehong method sila at makuha ang data.

SCENARIO 12: ID should be a number, so remove the quotes. Casting as integer makes it clearer and safer.

SCENARIO 13: Para safe, dapat may WHERE clause sa UPDATE, otherwise maa-update lahat ng data.

SCENARIO 14: Check array keys and make sure they're quoted, and strings should have quotes too. Pag tama, okay na ang SQL.

SCENARIO 15: If the page number is way too big, lalaki nang sobra ang offset at puwedeng bumagsak ang database. Limit and validate para safe.